multiple threads to separate concerns and as such requires you to explicitly decide how to divide the tasks between threads.

4.5. Summary

Synchronizing operations between threads is an important part of writing an application that uses concurrency: if there's no synchronization, the threads are essentially independent and might as well be written as separate applications that are run as a group because of their related activities. In this chapter, I've covered various ways of synchronizing operations from the basic condition variables, through futures, promises, and packaged tasks. I've also discussed ways of approaching the synchronization issues: functional-style programming where each task produces a result entirely dependent on its input rather than on the external environment, and message passing where communication between threads is via asynchronous messages sent through a messaging subsystem that acts as an intermediary.

Having discussed many of the high-level facilities available in C++, it's now time to look at the low-level facilities that make it all work: the C++ memory model and atomic operations.